

Kubernetes Services

enabling clients to discover and talk to pods

Kubernetes in Action · Chapter 5

minikube 4.5.2 (1 CP + 3 Worker) · Calico CNI · Podman

오늘 다룰 내용

#	주제	핵심
1	ClusterIP	내부 로드밸런싱
2	Session Affinity	클라이언트 고정
3	Service Discovery	환경변수 vs DNS
4	NodePort	외부 노출
5	Endpoints	Service의 내부 구조
6	Readiness Probe	준비 안 된 Pod 보호
7	Headless Service	개별 Pod 찾기
8	Ingress	L7 HTTP 라우팅

왜 Service가 필요한가?

Pod IP를 직접 쓰면 생기는 문제

Pod IP는 안정적이지 않다

Before
Client --> 10.108.1.4 --> Pod A 

Pod 재시작 후
Client --> 10.108.1.4 --> ???  사라짐
New Pod --> 10.108.2.5 (다른 IP)

- Pod은 언제든지 삭제되고 새로 생성될 수 있다
- 새 Pod은 다른 IP를 받는다
- 같은 역할의 Pod이 여러 개일 수 있다
- 클라이언트가 Pod 목록을 직접 추적? → 코드가 K8S에 강하게 결합

Service = 고정된 접근 지점



Service는 불안정한 Pod 집합을 안정적인 네트워크 엔드포인트로 추상화한다.

1. ClusterIP Service

클러스터 내부 로드밸런싱

ClusterIP — 가장 기본적인 Service

클러스터 내부에서만 접근 가능한 가상 IP를 제공한다.

```
apiVersion: v1
kind: Service
metadata:
  name: kubia
spec:
  type: ClusterIP      # 기본값
  ports:
    - port: 80         # Service 포트
      targetPort: 8080  # Pod이 듣는 포트
  selector:
    app: kubia         # 이 라벨의 Pod들을 묶는다
```

요청 흐름:

```
Client --> Service IP:80 --> Pod IP:8080
```

L4 수준 분산 — HTTP path나 cookie를 보지 않고, **TCP 연결** 단위로 백엔드 선택



데모: ClusterIP 로드밸런싱

```
kubectl apply -f kubia-replicaset.yaml # Pod 3개  
kubectl apply -f kubia-svc-clusterip.yaml # ClusterIP Service
```

```
kubectl get svc kubia  
# NAME      TYPE          CLUSTER-IP      PORT(S)    AGE  
# kubia     ClusterIP     172.20.227.100  80/TCP     5s
```

```
for i in $(seq 1 10); do  
  kubectl exec kubia-59xhn -- wget -qO- http://172.20.227.100  
done
```

```
You've hit kubia-wjr92 ← Pod 1  
You've hit kubia-59xhn ← Pod 2  
You've hit kubia-59xhn  
You've hit kubia-wjr92  
You've hit kubia-dltvv ← Pod 3  
You've hit kubia-dltvv
```

같은 IP에 10번 요청 → 3개 Pod 모두에 분산

2. Session Affinity

같은 클라이언트 → 같은 Pod

Session Affinity

설정	동작
None (기본)	매 연결마다 다른 Pod 가능
ClientIP	같은 클라이언트 IP → 같은 Pod 고정

K8S Service는 L4 수준이므로 쿠키 기반 세션은 지원하지 않는다

```
# ClientIP 적용
kubectl patch svc kubia -p '{"spec":{"sessionAffinity":"ClientIP"}}'

# 복원
kubectl patch svc kubia -p '{"spec":{"sessionAffinity":"None"}}'
```



데모: Session Affinity 비교

ClientIP 적용 후

```
You've hit kubia-59xhn
You've hit kubia-59xhn
You've hit kubia-59xhn
You've hit kubia-59xhn
You've hit kubia-59xhn
(x10 전부 동일!)
```

None 복원 후

```
You've hit kubia-59xhn
You've hit kubia-wjr92
You've hit kubia-wjr92
You've hit kubia-dltvv
You've hit kubia-59xhn
(다시 분산)
```

설정 한 줄 차이로 동작이 완전히 달라진다

3. Service Discovery

앱이 Service를 찾는 두 가지 방법

환경변수 vs DNS

방식	메커니즘	Service가 먼저 있어야?
환경변수	Pod 생성 시 K8S가 주입	예
DNS	CoreDNS가 실시간 해석	아니오

환경변수 예시:

```
KUBIA_SERVICE_HOST=172.20.227.100  
KUBIA_SERVICE_PORT=80
```

DNS 예시:

```
kubia.default.svc.cluster.local → 172.20.227.100
```

데모: Service Discovery

기존 Pod (Service보다 먼저 생성됨):

```
kubectl exec kuba-59xhn -- env | grep KUBIA  
# (출력 없음!) ← 환경변수 없음
```

새 Pod (Service 이후에 생성):

```
kubectl run tmp-test --image=busybox:1.36 --restart=Never --rm -it \  
  -- sh -c 'env | grep KUBIA'  
# KUBIA_SERVICE_HOST=172.20.227.100 ← 있다!  
# KUBIA_SERVICE_PORT=80
```

DNS는 순서 무관:

```
kubectl exec kuba-59xhn -- wget -q0- http://kuba  
# You've hit kuba-wjr92 ← 기존 Pod에서도 동작!
```

실무에서는 **DNS**를 쓰세요. 순서 의존성이 없습니다.

4. NodePort

클러스터 외부에서 접근하기

NodePort — 모든 노드에 포트를 연다

클러스터의 4개 노드 모두에 동일 포트(30123)가 열림:

192.168.49.2:30123	(minikube)	← 열림
192.168.49.3:30123	(m02)	← 열림
192.168.49.4:30123	(m03)	← 열림
192.168.49.5:30123	(m04)	← 열림

Pod이 없는 노드로 요청해도 kube-proxy가 전달:

외부 → Node A:30123 → kube-proxy → Pod@Node B
(Pod 없음) (Pod 있음)

NodePort — 세 포트의 관계

```
spec:
  type: NodePort
  ports:
    - port: 80          # ① Service 포트
      targetPort: 8080  # ② Pod이 듣는 포트
      nodePort: 30123   # ③ 노드에 열리는 포트
```

포트	역할	누가 접근?
port (80)	Service 자체	클러스터 내부 Pod
targetPort (8080)	컨테이너 수신	Service → Pod
nodePort (30123)	노드 외부 노출	외부 클라이언트

External Client --30123--> Node --80--> Service --8080--> Pod



데모: NodePort 외부 접근

```
kubectl apply -f kubia-svc-nodeport.yaml

kubectl get svc kubia-nodeport
# NAME          TYPE          CLUSTER-IP      PORT(S)          AGE
# kubia-nodeport NodePort       172.20.183.229   80:30123/TCP     0s

minikube service kubia-nodeport --url
# http://127.0.0.1:63657
```

```
# 호스트 머신(클러스터 외부)에서 직접 접근!
for i in $(seq 1 5); do curl -s http://127.0.0.1:63657; done
```

```
You've hit kubia-wjr92      ← 외부에서 접근 성공!
You've hit kubia-59xhn
You've hit kubia-dltvv
```

지금까지는 클러스터 안에서만 됐는데, 로컬 터미널에서 직접 curl이 됩니다

5. Endpoints

Service가 Pod을 찾는 내부 구조

Endpoints — Service와 Pod 사이의 연결고리

Service는 Pod에 직접 트래픽을 보내지 않는다. 중간에 **Endpoints**가 있다.

```
Service (kubia)
  |
  | selector: app=kubia
  v
Endpoints (kubia)           ← 자동 생성/갱신
- 172.21.59.133:8080
- 172.21.205.197:8080
- 172.21.120.67:8080
```

Pod이 추가/삭제되면 Endpoints가 실시간으로 갱신된다.



데모: Endpoints 실시간 갱신

```
# 현재 (3개)
kubectl get endpoints kubia
# 172.21.151.12:8080, 172.21.205.200:8080, 172.21.59.137:8080

# 5개로 스케일
kubectl scale rs kubia --replicas=5
kubectl get endpoints kubia
# 172.21.120.71:8080, ... + 2 more          ← 5개로 증가!

# 2개로 스케일
kubectl scale rs kubia --replicas=2
kubectl get endpoints kubia
# 172.21.205.200:8080, 172.21.59.137:8080    ← 2개로 감소!
```

Pod 수를 바꿀 때마다 **Service** 설정을 건드리지 않아도 Endpoints가 자동 갱신

6. Readiness Probe

준비 안 된 Pod에 트래픽 차단

Liveness vs Readiness

항목	Liveness	Readiness
질문	"컨테이너가 죽었나?"	"트래픽 받을 준비 됐나?"
실패 시	컨테이너 재시작	Endpoints에서 제거
RESTARTS	증가	변화 없음
용도	deadlock, hang 감지	초기화, DB 연결 대기, shutdown

Readiness 실패 시 Pod은 살아 있지만 트래픽만 차단

```
readinessProbe:
  exec:
    command: ["ls", "/var/ready"] # 파일 있으면 Ready
  periodSeconds: 3
```

데모: Readiness Probe

1) 생성 직후 — `/var/ready` 파일 없음 → 전부 Not Ready:

```
kubia-readiness-8phbx    0/1    Running    ← Not Ready
kubia-readiness-gbjf7    0/1    Running
kubia-readiness-lznjb    0/1    Running
```

Endpoints: (비어 있음!)

2) 파일 생성 → 1개만 Ready 전환:

```
kubectl exec kubia-readiness-8phbx -- touch /var/ready
```

```
kubia-readiness-8phbx    1/1    Running    ← Ready!
Endpoints: 172.21.205.201:8080    ← 이 Pod만 등록
```


데모: Readiness Probe (계속)

3) 파일 삭제 → 다시 Not Ready:

```
kubectl exec kubia-readiness-8phbx -- rm /var/ready
```

```
kubia-readiness-8phbx    0/1    Running    RESTARTS=0    ← 재시작 없음!  
Endpoints: (다시 비어 있음)
```

Liveness 실패

- 컨테이너 재시작
- RESTARTS 증가

Readiness 실패

- Pod 유지, 트래픽만 차단
- **RESTARTS = 0**

Liveness와 달리 **Pod**을 죽이지 않는다. Endpoints에서만 빠진다.

7. Headless Service

DNS로 개별 Pod IP 찾기

Headless Service — `clusterIP: None`

일반 Service: DNS → ClusterIP 1개 → kube-proxy가 Pod 선택

Headless Service: DNS → Pod IP 여러 개 → 클라이언트가 직접 선택

```
spec:
  clusterIP: None      # ← Headless의 핵심
  selector:
    app: kubia
```

언제 필요한가?

- DB Primary/Replica 구분 (PostgreSQL, MySQL)
- Kafka 파티션 브로커 지목
- Elasticsearch peer discovery
- 분산 캐시 노드 탐색



데모: 일반 Service vs Headless DNS 비교

```
kubectl run dns-test --image=busybox:1.36 --restart=Never --rm -it -- sh -c '  
  nslookup kuba.default.svc.cluster.local  
  nslookup kuba-headless.default.svc.cluster.local'
```

일반 Service

```
kuba.default.svc.cluster.local  
→ 172.20.227.100
```

ClusterIP 1개

Headless Service

```
kuba-headless.default...  
→ 172.21.205.200  
→ 172.21.59.137  
→ 172.21.151.15
```

Pod IP 3개 직접 반환

일반 = IP 1개, Headless = Pod IP 전체. 분산 시스템의 peer discovery에 필수.

8. Ingress

L7 HTTP 라우팅

Ingress — 하나의 진입점으로 여러 Service

NodePort/LB는 서비스마다 별도 포트/IP가 필요하다.

NodePort (서비스마다 별도 포트)

```
:30123 → 쇼핑몰  
:30124 → API  
:30125 → 관리자
```

서비스 10개 = LB 10개 = 비용 증가

Ingress (하나의 진입점)

```
shop.example.com      → 쇼핑몰  
shop.example.com/api  → API  
admin.example.com     → 관리자
```

LB 1개로 전부 처리

Ingress — Service와의 차이

항목	Service (NodePort)	Ingress
계층	L4 (TCP/UDP)	L7 (HTTP/HTTPS)
라우팅 기준	IP + Port	Host + Path
외부 진입점	서비스마다 별도	1개로 여러 서비스
TLS	직접 지원 안 함	TLS termination

```
rules:  
- host: kuba.example.com  
  http:  
    paths:  
    - path: /foo → kuba Service  
    - path: /bar → kuba-bar Service
```



데모: Ingress Path 기반 라우팅

```
kubectl apply -f kuba-ingress.yaml  
kubectl port-forward -n ingress-nginx svc/ingress-nginx-controller 8080:80 &
```

```
# /foo → kuba Service  
curl -s -H "Host: kuba.example.com" http://localhost:8080/foo  
# You've hit kuba-59xhn  
# You've hit kuba-wjr92  
  
# /bar → kuba-bar Service  
curl -s -H "Host: kuba.example.com" http://localhost:8080/bar  
# You've hit kuba-bar-5lg67  
# You've hit kuba-bar-5lg67
```

같은 호스트, 같은 포트인데 **Path** 한 글자 차이로 다른 서비스에 도달

정리

5장 핵심 요약

Service 타입 비교

타입	외부 접근	주요 용도
ClusterIP	×	내부 서비스 간 통신
NodePort	✓ (노드IP:포트)	단순 외부 노출
LoadBalancer	✓ (외부 IP)	클라우드 환경
Headless	—	Pod 개별 discovery
+ Ingress	✓ (Host/Path)	L7 HTTP 라우팅

핵심 비교

	Liveness	Readiness	환경변수	DNS
실패 시	재시작	Endpoints 제거	—	—
순서 의존	—	—	✓	×
실무 권장	필수	필수	×	✓

한 문장 요약

Service는 변하는 Pod 집합을 안정적인 이름과 IP로 추상화하고,
Ingress는 여러 Service를 하나의 진입점에서 HTTP 규칙으로 분기한다.

Q&A — 예상 질문

"Service IP(ClusterIP)는 어디에 있는 건가요?"

→ 실제 네트워크 인터페이스에 붙은 IP가 아니라, kube-proxy가 iptables 규칙으로 만든 가상 IP. `ping` 이 안 되는 이유.

"NodePort를 운영에서 직접 쓰나요?"

→ 거의 안 씀. 앞에 LoadBalancer나 Ingress를 두는 것이 표준. 30000-32767 포트 범위는 사용자 친화적이지 않음.

"Headless Service는 언제 쓰나요?"

→ DB(Primary/Replica), Kafka(파티션 브로커), Elasticsearch(peer discovery) 등 각 Pod을 개별적으로 찾아야 하는 Stateful 시스템.

"Ingress Controller 없이 Ingress만 만들면?"

→ 아무 일도 안 일어남. Ingress = 규칙, Controller = 실행체.

감사합니다

Questions?